APARTMENT SOCIETY MANAGEMENT SYSTEM

End-to-end implementation walkthrough
Specification for the development agent

Layer	Locked technology
Frontend	Next.js App Router + TypeScript
Styling	Tailwind CSS
Backend	Next.js Route Handlers (Node.js runtime)
Relational database	MySQL + Prisma ORM
Document database	MongoDB + Mongoose
Validation	Zod
Architecture	Responsive full-stack web application

Purpose: A bounded, testable specification. The agent must build a working application with real persistence, not static mock screens.

Version 1.0 | 30 October 2026

1. Project objective and problem statement

Problem statement. Apartment societies coordinate apartment and resident records, monthly maintenance payments, parking allocation, vehicle information, staff, complaints, visitors and society communications. When these operations are handled through paper registers, spreadsheets or scattered messaging groups, records become difficult to maintain, retrieve and audit. The proposed Apartment Society Management System centralizes these operations in a web application backed by MySQL and MongoDB.

The system helps administrators maintain society records, residents submit and track complaints, maintenance staff act on assigned work, and security personnel record visitor entry and exit.

Primary outcomes

• Centralized apartment and resident records.

• Visibility into paid and pending maintenance payments.

• Parking-space allocation and availability tracking.

• Complaint submission, assignment, status tracking and activity history.

• Visitor entry/exit records searchable by apartment and date.

• Notices, announcements and resident feedback.

• Dashboard summaries derived from live database data.

Project boundary: This is an academic demonstration, not a production-grade accounting, access-control or security platform.

2. Technology and architecture

Layer	Decision / rule
Frontend	Next.js App Router and TypeScript; reusable pages, components, forms and tables.
Styling	Tailwind CSS only; responsive desktop-first interface.
API	Next.js Route Handlers; all database access stays server-side.
MySQL	Prisma ORM for relational entities, constraints and transactions.
MongoDB	Mongoose for flexible documents and activity history.
Validation	Zod for request bodies, route params and environment variables.
Charts	Optional; chart only real aggregates, never hard-coded metrics.

Request flow: Browser UI → Next.js Route Handler → validation/service layer → Prisma/MySQL or Mongoose/MongoDB. Do not connect to databases from browser code.

Suggested project structure

```
src/
  app/
    (auth)/login/
    (dashboard)/
      dashboard/ apartments/ residents/ staff/
      payments/ parking/ vehicles/ complaints/
      visitors/ notices/ feedback/
    api/
      auth/ apartments/ residents/ staff/
      payments/ parking/ vehicles/ complaints/
      visitors/ notices/ feedback/
  components/ layout/ ui/ forms/ tables/
  lib/ prisma.ts mongodb.ts auth.ts
  server/ services/ repositories/
prisma/ schema.prisma migrations/
models/                # Mongoose models
```

3. Users and permissions

Role	Permitted functionality
Administrator	Manage apartments, residents, staff, payments, parking, vehicles, complaints, notices and reports; assign complaints; manage acc

Role	Permitted functionality
Resident	View own profile/apartment, own payment status, parking/vehicles; create and track own complaints; read notices; submit feedback
Maintenance staff	View complaints assigned to them; update permitted status; add work notes.
Security personnel	Register visitor entry/exit and search permitted visitor history. No payment access.

Authentication decision: Implement real sign-in and server-side role checks. Add a supporting MySQL *UserAccount* table linked to a resident or staff record where applicable; administrator accounts may be standalone. The teammate preparing the ER diagram and relational schema must include this supporting table. Store password hashes only. Use secure sessions and demo accounts with setup-generated passwords.

HARD RULE: hiding a button is not authorization. Every protected API must check both role and record ownership.

4. MySQL relational model

Preserve these seven agreed core entities and their meanings. Add timestamps, indexes and constraints where appropriate; do not silently rename or repurpose the tables.

Table	Purpose / relationships
Apartment	One row per flat; parent of Resident, ParkingSpace and MaintenancePayment.
Resident	Resident profile; apartment_id references Apartment; OWNER or TENANT.
Staff	Staff profile, role, contact information and salary.
ParkingSpace	Parking slot; apartment_id references Apartment; slot type and allocation status.
Vehicle	Vehicle belongs to a resident and may reference a parking slot.
MaintenancePayment	Monthly payment for an apartment; amount, month, payment date and PAID/PENDING status.
Complaint	Complaint raised by a resident; optionally assigned to staff; OPEN/IN_PROGRESS/RESOLVED.

Relationship and integrity rules

• Apartment 1-to-many Resident, ParkingSpace and MaintenancePayment.

• Resident 1-to-many Vehicle and Complaint.

• Staff 1-to-many assigned Complaint; complaint may be unassigned.

• A parking slot cannot be allocated to multiple active vehicles; vehicle and slot types must be compatible.

• Use DECIMAL for money, not FLOAT. Add indexes to foreign keys and commonly filtered columns.

• Consider unique (apartment_id, payment_month) to prevent duplicate monthly bills.

• Use transactions for multi-row MySQL operations; avoid deleting records with payment/complaint history.

The lab's seven-table seed script is the baseline. Validate the final schema against the team's ER diagram and document any migration the agent adds.

5. MongoDB document model

MongoDB is not a duplicate copy of MySQL. Store flexible, event-like documents and reference relational entities by their integer IDs.

Collection	Suggested fields / behavior
visitors	visitorName, phone?, purpose, apartmentId, hostResidentId, vehicleNumber?, entryTime, exitTime?, recordedByUserId, status.
notices	title, body, category, priority, publishedAt, expiresAt?, createdByUserId, audience.
complaint_activity	complaintId, actorUserId, actorRole, action, oldStatus?, newStatus?, note?, createdAt. Append-only.
feedback	residentId, apartmentId, category, rating, message, submittedAt, status.
announcements	title, message, audience, createdByUserId, publishedAt, expiresAt?. May instead be combined with notices using documentTyp

• Index visitors by apartmentId + entryTime; complaint_activity by complaintId + createdAt; notices by publishedAt.

• Store timestamps in UTC and display local time in the UI.

• Visitor exit updates exitTime/status on the matching document.

• Append an activity document for complaint assignment, status changes and work notes.

• If a MySQL write succeeds but its MongoDB audit write fails, report the partial failure; never claim both succeeded.

6. Pages and required UI

Page	Required behavior
Login	Email/username, password, validation, useful errors and role-aware redirect.
Dashboard	Live counts: apartments, residents, pending payments, open complaints, available parking, visitors today.
Apartments	Search/list, create/edit/view flat, linked residents and payment summary.
Residents	Search/list, create/edit/view; apartment association and OWNER/TENANT.
Staff	List, create/edit and view staff role/contact.
Payments	Monthly records, paid/pending filters, record payment, apartment/month search and totals.
Parking	Slot list, available/allocated filters, allocate/release slot.
Vehicles	Register/update/deactivate vehicle; associate resident and valid parking slot.
Complaints	Submit, filter, assign staff, update status, view chronological activity.
Visitors	Register entry, record exit, search by apartment/date/name.
Notices	Create, publish, edit/archive, list by audience/date.
Feedback	Resident submission; admin review and status update.

Every page needs loading, empty, success and error states. Add confirmation before destructive actions, accessible labels, pagination for long lists, and real search/filter behavior. No dead buttons or fake dashboard values.

7. End-to-end workflows

Maintenance payment

• Admin selects apartment and billing month, enters amount and status.

• Require payment date when PAID; allow null payment date when PENDING.

• Reject duplicate monthly bills for one apartment.

• Residents can view only their apartment's payment history and pending balance.

• Dashboard totals/counts must be computed from MySQL.

Complaint lifecycle

```
Resident submits complaint
        -> OPEN
Admin assigns staff
        -> IN_PROGRESS
Staff adds work note / updates status
        -> RESOLVED
```

• Create Complaint in MySQL and initial activity in MongoDB.

• On assignment/status change, update MySQL and append MongoDB activity.

• Resident sees own complaints; assigned staff sees assigned work; admin sees all.

• Validate allowed status transitions.

Parking and vehicle

• Admin creates slots and marks AVAILABLE or ALLOCATED.

• Allocate a compatible available slot to a resident's vehicle.

• Prevent double allocation and incompatible vehicle/slot types.

• Release operation updates vehicle/slot records consistently.

Visitor entry and exit

• Security records visitor name, purpose, destination apartment and entry time.

• Record the logged-in security account as the person who registered entry.

• On exit, find active visit and record exit time.

• Search history by apartment and date range.

Notices and feedback

• Admin publishes notice with audience and optional expiry.

• Residents see only notices intended for their audience; expired notices are not current.

• Residents submit feedback; admin reviews and updates status.

8. API checklist

Resource	Minimum API actions
Auth	POST login; POST logout; GET current session.
Apartments / Residents / Staff	GET list/detail; POST create; PATCH update; guarded delete/deactivation.
Payments	GET list/detail/summary; POST create; PATCH update or mark paid.
Parking	GET list; POST create; PATCH allocate/release/update.
Vehicles	GET list; POST register; PATCH update; DELETE/deactivate.
Complaints	GET list/detail; POST create; PATCH assign/status; POST activity note.
Visitors	GET search/list; POST entry; PATCH exit.
Notices	GET published/list; POST create; PATCH edit/publish/archive.
Feedback	GET list; POST submit; PATCH review status.

Use consistent JSON response/error formats and suitable HTTP status codes. Validate every payload. Never trust residentId, staffId or role supplied by the browser without verifying it against the authenticated session.

9. Agent implementation order

Phase	Tasks	Exit condition
0 Inspect	Inspect repo, package manager, existing code and environment; report assumptions and risks; no destructive reset.	Plans and files reviewed; no destructive reset.
1 Foundation	Create/verify Next.js TS/Tailwind, lint/typecheck, env validation, layout/navigation.	App boots; checks pass.
2 Databases	Configure Prisma/MySQL and Mongoose/MongoDB; migrations, models and reusable seeds.	Both DBs connect; seed works.
3 Auth/RBAC	UserAccount, password hashing, secure session, role and ownership middleware.	Each role signs in; forbidden calls rejected.
4 Core SQL	Apartments, residents, staff, payments, parking and vehicles.	CRUD and constraints tested.
5 Complaints	Submission, assignment, status changes and Mongo activity history.	Full lifecycle works.
6 Mongo modules	Visitor entry/exit, notices, feedback and search.	Documents persist and filters work.
7 Dashboard	Connect all screens to live APIs; summaries, filters, pagination and states.	No fake data or placeholder actions.
8 QA/docs	Test workflows, permissions, clean setup; finish README.	Teammate can run from README.

10. Seed data and demo

• Keep at least 10 records in each of the seven core MySQL tables, adapting the existing lab seed data to satisfy final constraints.

• Add demo UserAccount records for Administrator, Resident, Maintenance and Security.

• Seed MongoDB with enough documents to demonstrate visitors, notices, complaint activity and feedback; aim for 10 per collection where practical.

• Seeds must be repeatable and must not create duplicates on rerun.

• Use fictional names, contacts and vehicle registrations only.

• Provide a local demo reset/seed command and clearly warn that reset deletes demo data.

11. Security and quality requirements

• Keep secrets in .env.local; provide .env.example with placeholders only.

• Hash passwords; never log passwords or session tokens.

• Enforce role and record ownership on every protected API.

• Use Prisma parameterized operations; validate and normalize input.

• Never expose connection strings or privileged errors to the browser.

• Use safe deactivation where dependent records exist; preserve payment/complaint history.

• Handle database failures visibly; do not silently discard writes.

• Use MySQL transactions for multi-row operations; document cross-database consistency limitations.

• Run lint, TypeScript checks, production build and tests before handover.

12. Acceptance tests

Area	Pass condition
Clean setup	Fresh clone installs, configures env, migrates, seeds and starts using README.
Database	MySQL migrations and MongoDB models/indexes initialize.
Seed	Every core SQL table has at least 10 rows; seed can be rerun safely.
Payments	Duplicate apartment/month bill rejected; paid/pending totals correct.
Parking	Unavailable slot cannot be allocated; no double allocation.
Complaints	Resident submits; admin assigns; staff updates; resident sees status and timeline.
Visitors	Security records entry/exit; apartment/date search works.
Permissions	Resident cannot view another resident's private data; staff cannot access admin-only modules.
UI	All primary actions work; no mock metrics, dead links or broken forms.
Build	Lint, typecheck, production build and key workflow tests pass.

13. Final handover deliverables

• Complete source code in a clear project structure.

• Prisma schema and versioned MySQL migrations.

• Mongoose models and MongoDB indexes.

• Repeatable seed scripts for both databases and demo accounts.

• Working UI and APIs for all listed modules.

• Validation and server-side authorization for protected actions.

• README: prerequisites, environment variables, exact setup, migration, seed, run and demo-login instructions.

• Architecture summary, implemented-feature checklist, test results and known limitations.

14. Exact instruction to give the coding agent

Read this entire implementation guide before editing the repository. Treat it as the project specification and source of truth. First inspect the existing repository and report its structure, stack and current state. Then implement the system in the phases and order defined here. Do not replace working code or perform a destructive rewrite without explaining why. Do not build only mock UI: every listed module must use real API routes and persistent MySQL/MongoDB data. Preserve the seven agreed core MySQL entities, add the documented UserAccount support

for real role-based authentication, and keep document data in MongoDB without duplicating relational records. Enforce permissions on the server. Validate every workflow against the acceptance criteria. Work phase by phase; run lint, typecheck, build and tests after meaningful changes; fix errors before proceeding; and update the README and checklist as implementation progresses. If the existing ER diagram or relational schema conflicts with this guide, stop and report the exact conflict rather than silently inventing a third design.

15. Explicitly out of scope

• No online payment gateway; record payment status only.

• No biometric/face recognition, QR gate hardware or IoT integration.

• No real-time chat, video calling or complex notification service.

• No AI features or external paid APIs.

• No duplicate MongoDB copies of Apartments, Residents, Staff or Payments.

• No unrelated e-commerce, property sales or rental-listing modules.

• Do not claim production security/compliance.

Priority if time is limited: Finish login/roles, apartment/resident records, payments, parking, complaints, visitors, notices and feedback before decorative animations or visual polish.